

3.5.1

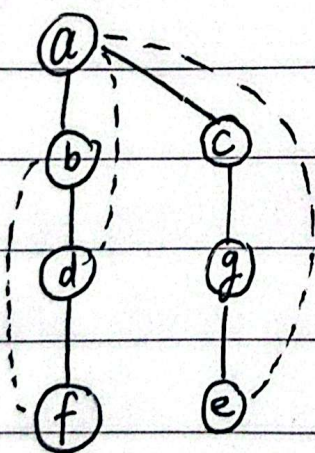
a. 邻接矩阵:

	a	b	c	d	e	f	g
a	0	1	1	1	1	0	0
b	1	0	0	1	0	1	0
c	1	0	0	0	0	0	1
d	1	1	0	0	0	1	0
e	1	0	0	0	0	0	1
f	0	1	0	1	0	0	0
g	0	0	1	0	1	0	0

邻接链表:

a	→ b → c → d → e
b	→ a → d → f
c	→ a → g
d	→ a → b → f
e	→ a → g
f	→ b → d
g	→ c → e

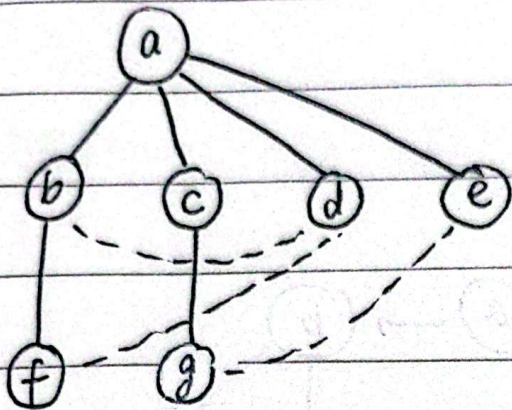
b. DFS, a 起点



顶点入栈顺序: $a_1, b_2, d_3, f_4, c_5, g_6, e_7$

顶点出栈顺序: $f_1, d_2, b_3, e_4, g_5, c_6, a_7$

3.5.4. BFS; a 起点,



4.1.10

两种算法的复杂度类别是一样的, 都是 $O(n^2)$,

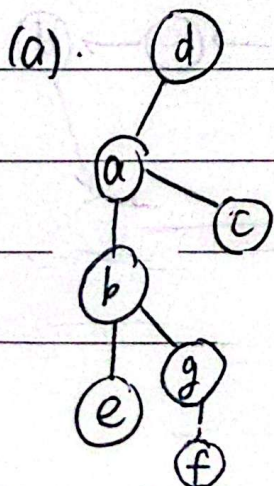
insertionsort 的内层循环包括 1 次赋值和 1 次索引递减操作;

insertionsort2 的内层循环包括 1 次 swap (相当于 3 次赋值) 和 1 次索引递减操作.

若忽略索引递减操作的时间, 则两者运行时间 insertionsort2 约为课本 insertionsort 的 3 倍.

4.2.1

DFS:



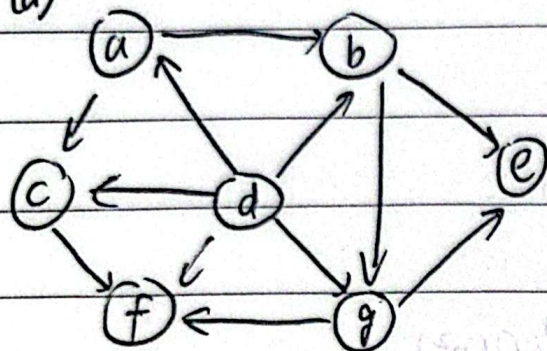
出栈: $e_1, f_2, g_3, b_4, c_5, a_6, d_7$

逆序为拓扑排序: d, a, c, b, g, f, e

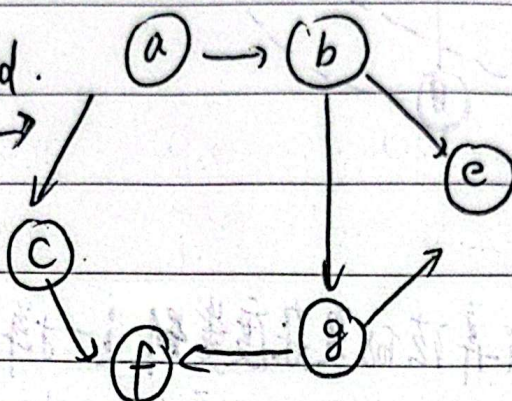
(b). 该图不是无环有向图, 有一个 $a \rightarrow b \rightarrow c \rightarrow d \rightarrow g \rightarrow e \rightarrow a$ 的环路, 故无解, 没有拓扑排序的解.

4.2.5 源删除算法:

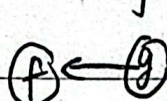
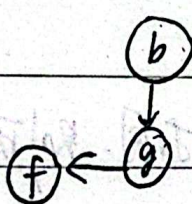
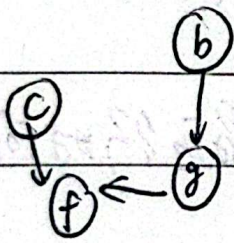
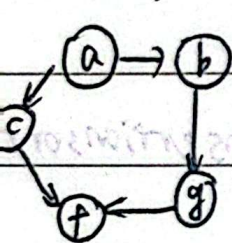
(a)



① 删 d.

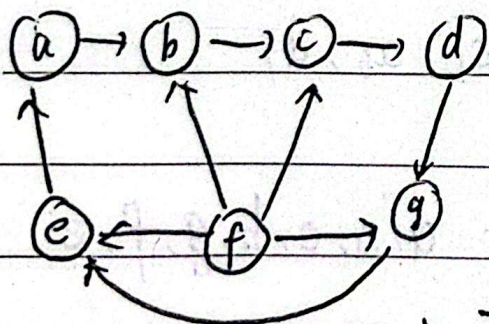


② 删 e. $\rightarrow$ ③ 删 a. $\rightarrow$ ④ 删 c. $\rightarrow$ ⑤ 删 b. $\rightarrow$ ⑥ 删 g. $\rightarrow$ ⑦ 删 f

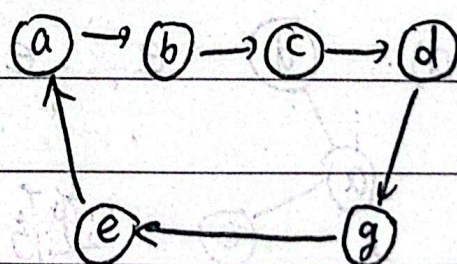


则其中一种拓扑解为 d, e, a, c, b, g, f.

(b).



删 f



环路, 无法再删除, 故无解.

4.3.2 {1, 2, 3, 4}

(a) 从底向上的最小变化法.

开始: 1

从右到左将2插入1: 12 21

从右到左将3插入12: 123 132 312

从左到右将3插入21: 321 231 213

从右到左将4插入123: 1234 1243 1423 4123

从左到右将4插入132: 4132 1432 1342 1324

从右到左将4插入312: 3124 3142 3412 4312

从左到右将4插入321: 4321 3421 3241 3214

从右到左将4插入231: 2314 2341 2431 4231

从左到右将4插入213: 4213 2413 2143 2134

(b) Johnson-Trotter 算法.

← ← ← ← ← ← ← ← ← ← ← ← ← ← ← ←
1 2 3 4 1 2 4 3 1 4 2 3 4 1 2 3

→ ← ← ← ← → ← ← ← ← → ← ← ← ← →
4 1 3 2 1 4 3 2 1 3 4 2 1 3 2 4

← ← ← ← ← ← ← ← ← ← ← ← ← ← ← ←
3 1 2 4 3 1 4 2 3 4 1 2 4 3 1 2

→ → ← ← → → ← ← → ← → ← → ← ← →
4 3 2 1 3 4 2 1 3 2 4 1 3 2 1 4

← → ← ← ← → ← ← ← ← → ← ← ← → ←
2 3 1 4 2 3 4 1 2 4 3 1 4 2 3 1

→ ← ← → ← → ← → ← ← → → ← ← → →
4 2 1 3 2 4 1 3 2 1 4 3 2 1 3 4

(c) 字典序算法.

1234	1243	1324	1342	1423	1432
2134	2143	2314	2341	2413	2431
3124	3142	3214	3241	3412	3421
4123	4132	4213	4231	4312	4321

4.3.6 建立集合 $A = \{a_1, a_2, \dots, a_n\}$ 的所有子集与长度为 n 的二进制 $b_1 b_2 \dots b_n$ 之间的一一对应关系, 将 b_i 与 a_{n-i+1} 之间关联, 若 $b_i = 1$, 表示 a_{n-i+1} 存在于子集, 若 $b_i = 0$, 表示 a_{n-i+1} 不存在于子集.

4.4.8 (a) 基于减治法中的减常因子算法:

(b) $C_{\text{worst}}(n) = C_{\text{worst}}(\lfloor n/3 \rfloor) + 2, C_{\text{worst}}(1) = 1.$

$n = 3^k$, 则 $C_{\text{worst}}(3^k) = C_{\text{worst}}(3^{k-1}) + 2.$

(c) $n > 1, C_{\text{worst}}(3^k) = C_{\text{worst}}(3^{k-1}) + 2$

$$= C_{\text{worst}}(3^{k-2}) + 2 \times 2$$

$$= C_{\text{worst}}(3^{k-3}) + 2 \times 3$$

...

$$= C_{\text{worst}}(3^{k-k}) + 2 \times k = 2k + 1$$

$n = 3^k$, 则 $C_{\text{worst}}(n) = 2 \log_3 n + 1$

(d) 折半查找最差效率:

二分: $C_{\text{worst}}(n) = \log_2 n + 1$

三重: $C_{\text{worst}}(n) = 2\log_3 n + 1$

$$(\log_2 n + 1) - (2\log_3 n + 1) = \log_2 n - 2\log_3 n$$

$$\frac{\log_2 n}{2\log_3 n} = \frac{1}{2} \log_2 3 \approx 0.79 < 1$$

故折半查找的最差效率更高,
两种算法的时间复杂度均为对数级

4.5.13

将待查找元素与第一行的最后一个元素比较。若待查找数字小于该数字, 则目标不在最后一列, 剔除该列以缩小查找范围; 若待查找数字大于该数字, 则目标不在第一行, 剔除该行以缩小查找范围。

对缩小后的矩阵重复以上步骤, 直到找到目标或矩阵变空。

时间复杂度为 $O(n)$